SmartPtr-C 2.0
Viva & Q&A Preparation

Every question your professor can ask, answered simply

How to use this: each question has a short **answer to say out loud** (in the green box) and, below it, the explanation behind it, so you understand it and are not just memorising.

Tags: CORE very likely to be asked · DEEP if he goes into detail · TRICKY questions meant to test you.

Part D answers his earlier question about the **hierarchy of freeing** in C. The last page lists things you should **not** claim.

Team Error-404 · Project 12

The 60-second explanation

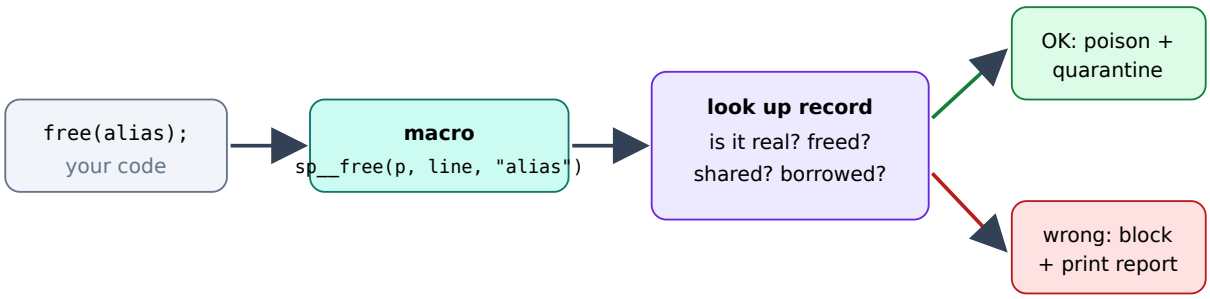
If he says "explain your project", say this. Practise it until it flows.

> **Say:** "*In C, a pointer is only an address. It does not remember who owns the memory or whether it was already freed. That causes double frees, overflows, use-after-free and leaks, and when they happen C either crashes without telling you the line, or silently corrupts memory.*
>
> *Our library has two layers. The first,* **Sentinel**, *needs no new functions: you include one header, and every* `malloc` *and* `free` *in that file goes through our code first. We keep a record of every block, so we can block a double free, detect overflows and writes after free, and report every leak with the exact line.*
>
> *The second layer is* **ownership**: *unique, shared and weak pointers like C++, plus every conversion between them, scope-based cleanup and borrowing. Every mistake becomes a readable report instead of a crash. We have 58 tests, and Valgrind, AddressSanitizer and ThreadSanitizer all report zero problems.*"

The three words to keep in your head

Word	Meaning in one line
Intercept	a macro turns your `free(p)` into *our* function before the real `free` runs
Record	for every block we keep a small note: address, size, owner kind, owner count, where it was made, its history
Quarantine	freed memory is not given back immediately; it is "frozen", filled with `0xDD` and watched

The whole flow in one picture



Part A · Fundamentals (he may test your basics)

A1 What is the stack and what is the heap? CORE

> **Say:** "*The stack holds local variables and is cleaned up automatically when a function returns. The heap is memory we ask for with malloc; it stays until we free it, so the programmer is responsible for it.*"

Our library only deals with the **heap**, because that is where the bugs are.

A2 What exactly does malloc return, and what does free need? CORE

> **Say:** "*malloc returns the starting address of a block. free needs exactly that same address, once. Giving it anything else, or giving it twice, is undefined behaviour.*"

A3 What is "undefined behaviour"? CORE

> **Say:** "*It means the C standard makes no promise about what happens. It may crash, may seem to work, or may corrupt memory and fail much later. That is why memory bugs are so hard to find.*"

Example: the second `free` in `raw_bug.c` happens to be caught by glibc ("double free detected in tcache 2"), but with different sizes or timing it can silently corrupt the heap instead.

A4 Name the memory bugs your project handles. CORE

Bug	One-line example
Double free	`free(p); free(p);`
Invalid free	`free(&local_var);`
Interior free	`free(p + 4);`
Buffer overflow	`malloc(5)` then writing 6 bytes
Use after free (write)	`free(p); p[0] = 1;`
Memory leak	malloc without free
Reference cycle	A owns B and B owns A

A5 What is a smart pointer? CORE

> **Say:** "*A pointer that also knows who owns the memory and frees it at the right time. C++ has unique_ptr, shared_ptr and weak_ptr. C has nothing like it, so we built it.*"

> **Python comparison:** every Python variable is already a "shared pointer". Python counts references and frees the object when the count reaches zero. Our shared pointer does the same counting, in C.

A6 What is a macro? CORE

> **Say:** "*A find-and-replace the preprocessor does before compiling.* `#define free(p) sp__free(p, ...)` *means every* `free(x)` *in the file is rewritten into a call to our function.*"

Part B · "Only original malloc and free — how does it work?"

This is what your team was unsure about. Read this part carefully.

B1 The user only calls normal malloc and free. How does your code even run? CORE

> **Say:** "*Through macros in `smartptr.h`. When the user includes it, `malloc`, `calloc`, `realloc`, `strdup` and `free` are redefined as macros that call our functions. The user's code does not change, but the compiler compiles calls to our functions.*"

```
/* inside smartptr.h */
#define malloc(size)  sp__malloc((size), SP_HERE)
#define free(ptr)     sp__free((ptr), SP_HERE, #ptr)
#define SP_HERE       ((sp_site){ __FILE__, __LINE__, __func__ })

/* what the user writes on line 42 */   free(alias);
/* what the compiler actually compiles */
sp__free(alias, (sp_site){"01_zero_api.c", 42, "main"}, "alias");
```

- `__FILE__`, `__LINE__`, `__func__` are filled in by the compiler, so we know the exact line.
- `#ptr` turns the argument into a string, so we know the variable's **name** ("alias").
- Inside our function we call the **real** `malloc` / `free`. Our own `.c` files do not include `smartptr.h`, so for them the names still mean the real functions.

B2 Does it "auto-correct" everything? TRICKY

> **Be careful here.** Don't say "it fixes every bug automatically". Say exactly what happens for each bug.

Bug	What we do	Why that is safe
Double free	**Prevent**	we see the block is already freed and never call the real free again
Invalid / interior free	**Prevent**	the address is not a block start in our table, so we refuse
free() on a shared object	**Correct**	we treat it as "drop one reference"; the other owners keep working
Release while borrowed	**Correct**	destruction is postponed until the borrower is done
Leak	**Report + reclaim**	at exit we report it and free it; during the run it still uses memory
Overflow, use-after-free write	**Detect + report**	the damage has already happened; we tell you exactly where so you can fix it

> **Say:** "*It prevents the bugs that can be prevented at the moment of the call (double, invalid and interior frees), handles ownership conflicts safely, and detects and explains the ones that can only be noticed afterwards, like overflows. Nothing crashes, and every problem is reported with its line.*"

B3 How does a double free get blocked if you call the real free? CORE

1. First `free(p)`: we find the record, mark it **FREED**, poison the bytes, and keep the block in quarantine. **We do not call the real free yet.**
2. Second `free(p)`: we look up `p`, the record says FREED, so we print the report and **return without calling anything**.

> **Say:** "*The real free is called only once, when the block leaves quarantine. A second free finds the record marked FREED and is stopped before it reaches the real free.*"

B4 What if a file doesn't include smartptr.h? TRICKY

> **Say:** "*Then that file uses the normal functions and is not protected. Including the header is the one requirement. A future version could use LD_PRELOAD to intercept at link level without recompiling.*"

B5 If memory isn't freed immediately, isn't that wasteful? DEEP

> **Say:** "*Slightly. The quarantine holds at most 4096 freed blocks or 16 MB, then releases the oldest. That limit is configurable with SENTINEL_QUARANTINE. It's a deliberate trade: a little memory in exchange for catching double frees and late writes.*"

Part C · How Sentinel works inside

C1 What is stored about each block, and where? CORE

A struct called `sp_record` (in `src/sp_internal.h`): address, size, id, type name, kind (raw / unique / shared), state (LIVE / FREED...), owner count, weak count, borrow count, destructor, where it was created and freed, and the last 12 events (malloc, move, share, drop...).

All records live in a **hash table** keyed by the block's address, so finding the record for any pointer is O(1) on average.

> **Python comparison:** it is a dictionary: `records[address] = {"size": 24, "kind": "unique", "owners": 1, ...}`

C2 Why keep the record beside the memory, not in front of it like many allocators? DEEP

> **Say:** "*So the pointer we return is exactly the pointer the real malloc returned. Then the same memory can be raw, unique or shared without moving it, and converting between kinds only changes the record.*"

C3 How do you detect a buffer overflow? CORE

We ask the real malloc for **16 extra bytes** after the user's block and fill them with a known pattern. On free, realloc, `sp_guard_check()` and at exit, we compare the pattern. If a byte changed, something wrote past the end. The position of the last changed byte tells us how far the overflow went.

```
[ user's 5 bytes ][ A5 8C 73 ... 16 sentinel bytes ]
  "Hello" + '\0' writes 6 bytes →  the first sentinel byte changes → "1 byte past the end"
```

> **Say:** "*Like a tripwire. We place known bytes right after each block; if they change, someone wrote past the end.*"

C4 What is quarantine? What did you mean by "freezing" memory? CORE

> **Say:** "*When you free a block, we 'freeze' it instead of giving it back: we fill it with the value 0xDD and keep it in a waiting line. While it is frozen, a second free is recognised, and if anything writes into it, the 0xDD pattern changes. When the block finally leaves the line we check the pattern, then really free it.*"

Why `0xDD`? It's an unusual value, unlikely to be written by normal code, and easy to spot ("dead") in a debugger.

C5 How do you find leaks? CORE

We register a function with `atexit()`, so it runs when `main` returns. Every record still LIVE at that moment was never freed: that's a leak. We group leaks by the line that allocated them, so "allocated 3 times from line 63" points to a missing free inside a loop.

C6 What are direct and indirect leaks? DEEP

We read every leaked block word by word. If a word equals the address of another leaked block, the first points to the second. A leaked block pointed to by another leaked block is **indirect**: it is lost only because its parent was lost. The rest are **direct**. Fix the direct ones and the indirect ones disappear too.

C7 How do you detect a reference cycle? DEEP

> **Say:** "*At exit we build a graph: leaked blocks are nodes and pointers between them are edges. Then we run Tarjan's strongly connected components algorithm. Any group of two or more blocks that can reach each other is a cycle, and we print the loop, like #3 Node → #4 Node → #3 Node.*"

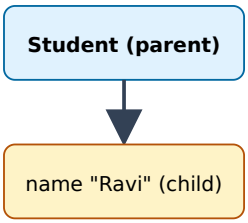
Tarjan's algorithm is a standard depth-first-search algorithm that finds all such groups in one pass (linear time).

C8 How does the report know the variable name and the line? CORE

From the macro: `__LINE__`, `__FILE__`, `__func__` and `#ptr` (the argument's source text). Every operation also stores an event in the block's history, which becomes the "timeline" in the report.

Part D · "We cannot control the hierarchy while freeing in C"

What he means: objects contain pointers to other objects (a tree, a student with a name string, a list). When you free them, the **order** matters, and C gives no help:



Parent freed first → child address is lost → LEAK

Child freed while parent still uses it → DANGLING pointer

Child shared by two parents → who frees it? → DOUBLE FREE

D1 So how DO you control the order of freeing? CORE

> **Say:** "*Every smart object can have a destructor, a function we call before freeing the object. Inside it, the object releases its children first. Only after the destructor returns do we free the parent. So the order is always children first, then parent, and the programmer writes that rule once, not at every free.*"

```
static void student_dtor(void *obj) {
    Student *s = obj;
    free(s->name);          // 1. child first
}                           // 2. then the library frees the Student itself

Student *s = sp_make_unique_d(Student, student_dtor);
sp_drop(&s);                // runs student_dtor, then frees s
```

> **Python comparison:** it's like `__del__`, a cleanup method that runs automatically before the object disappears.

D2 What about a whole tree, many levels deep? CORE

Each node's destructor drops its own children. Dropping the root therefore starts a **cascade** down the tree. We tested this exactly:

```
root ─┬─ left ── left.left            sp_drop(&root);  prints:
      └─ right  (also owned by 'keep')      destroying root
                                            destroying left
                                            destroying left.left
                                            right still alive (owners 1)   ← shared with 'keep'
                                            right's parent expired: yes
```

> **Say:** "*Freeing is decided by ownership, not by position in the tree. A child is destroyed only when its last owner lets go. If a child is shared with someone else, it survives its parent. That's exactly the hierarchy problem plain C can't solve.*"

D3 Children often point back to their parent. Doesn't that cause problems? DEEP

Yes. If the child **owns** the parent and the parent owns the child, that's a cycle and neither is ever freed (this happens with C++ shared_ptr too). The rule: **down = owning, up = weak**. The parent link is an `sp_weak`, which observes without owning. If someone forgets and creates a cycle anyway, Sentinel finds it at exit and prints the loop.

> **Say:** "*Parent-to-child links own, child-to-parent links are weak. And if a cycle is made by mistake, we detect it with Tarjan's algorithm at exit.*"

D4 What if code holds a pointer to a child after the parent freed it? DEEP

- **Weak pointer:** `sp_weak_expired(&w)` tells you it's gone; `sp_weak_lock` returns NULL instead of a dangling pointer.
- **Borrowing:** if the code borrowed it (`sp_borrow`), the destruction is **postponed** until `sp_unborrow`, so the pointer stays valid.
- **Plain pointer:** using it is caught as a use-after-free (write) or "object already destroyed" (smart call).

D5 And plain malloc code with no destructors? Can you free children automatically? TRICKY

> Honest answer: **no**. In plain C we can't know which fields inside a struct are pointers the struct owns.

What we **do**: if the parent is freed but the child isn't, the child becomes a leak, and at exit we report it with the line that allocated it ("tree.c:31, 7 bytes, never released") and reclaim it. To get automatic, ordered freeing, give the object a destructor (D1) or use `sp_auto`.

> **Say:** "*C has no information about which fields are owned. We solve that with destructors and ownership. For plain code, we report the forgotten child with its exact line and reclaim it at exit.*"

Part E · Smart pointers

E1 Unique vs shared vs weak? CORE

Kind	Rule	Freed when
unique	exactly one owner	that owner drops it
shared	owner count	the count reaches 0
weak	observes, does not own	never frees; can tell if the object is gone

E2 Your smart pointer is just `Student *`. Where is the "smart" part? DEEP

> **Say:** "*The ownership lives in the block's record, not in the pointer. The pointer stays an ordinary pointer, so `->`, arrays and existing functions all work. Every ownership operation looks up the record and updates it.*"

E3 How does move guarantee only one owner? CORE

`registrar = sp_move(&admission);` takes the **address of the variable**, returns the pointer and sets the old variable to NULL. Since the old variable is NULL, it can no longer free the object.

E4 Why does `q = p;` cause a double free but `q = sp_share(p);` doesn't? CORE

`=` only copies the address; the owner count stays 1, so dropping both is two releases. `sp_share` adds 1 to the count, so the object survives until both drop it.

E5 How does a weak pointer know the object died, even after the memory is reused? DEEP

The weak handle stores the **record** and the block's unique **id**, not just the address. While any weak handle exists, the record is kept alive (memory freed, record marked EXPIRED). So the answer to "is it alive?" is always correct and never a dangling read.

E6 Which conversions exist, and which can C++ not do? CORE

raw → unique (`sp_adopt`), raw → shared (`sp_adopt_shared`), unique → shared (`sp_to_shared`), shared → unique (`sp_to_unique`), unique → raw (`sp_release`), unique/shared → weak (`sp_weak_from`), weak → shared (`sp_weak_lock`).

Beyond C++: shared → unique (allowed only when there's exactly one owner), and weak observers of a *unique* object (`sp_weak_peek`).

E7 How does `sp_auto` free automatically? DEEP

It's GCC/Clang's `__attribute__((cleanup(fn)))`. The compiler inserts a call to our function whenever the variable goes out of scope, on every path: normal end, `return`, `break`. Our function drops or frees whatever the variable holds.

> **Python comparison:** `with open(...) as f:`. The file closes when the block ends, whatever happens.

E8 What is borrowing? DEEP

`sp_borrow(p)` says "I'm using this". If the owner drops it meanwhile, we don't destroy it; we mark it "pending" and destroy it at the final `sp_unborrow`. This prevents a dangling pointer in the borrower.

Part F · Outer, comparison and tricky questions

F1 Isn't this just Valgrind or AddressSanitizer? TRICKY

> **Say:** "*Those are excellent detectors, and we use them to verify our own code. But they only detect. They don't prevent the double free, they don't know about ownership, and they can't do unique/shared/weak or conversions. Ours is a library inside the program: it prevents some bugs at the call, explains them in ownership terms, and adds smart pointers. Valgrind also makes programs 20–50× slower and needs a separate tool run.*"

F2 Why not just use C++? TRICKY

Huge amounts of code (operating systems, embedded systems, drivers) must stay in C. C++ smart pointers also don't help existing `malloc` / `free` code, don't detect cycles, and make a double free undefined behaviour instead of a reported error.

F3 Is this garbage collection? TRICKY

> **Say:** "*No. A garbage collector frees memory by itself while the program runs. We don't. Freeing still happens at defined points: free, sp_drop, the last shared owner, or the end of an sp_auto scope. We only scan memory at exit to explain leaks and cycles.*"

F4 What does it cost? CORE

Measured with `make bench`: plain malloc/free about 9 ns, through Sentinel about 270 ns (about 30× slower); share/drop about 115 ns. That's the price of the record, poisoning and checks, and it's normal for a checking tool. It's meant for development and testing, and `SP_NO_INTERCEPT` turns interception off per file.

F5 Is it thread-safe? CORE

Yes. One mutex protects the table. It's released while a destructor runs, because the destructor may call free itself. The test with 8 threads × 5,000 rounds gives an exact owner count, and ThreadSanitizer reports no data races. A faster design would use per-bucket locks and atomic counters.

F6 What can it NOT detect? TRICKY

- **Reading** freed memory (only writes are detected).
- Writes **before** the start of a block; overflows further than 16 bytes past the end.
- A double free after the block left quarantine (after 4096 more frees).
- Code in files that don't include the header.

> **Say:** "*We document these limits openly in the README. Guard pages using mprotect would catch reads after free; that's future work.*"

F7 What if someone passes `free` as a function pointer? TRICKY

A function-like macro only expands when followed by `(`, so `free` written alone is the real function and bypasses us. That's a documented limitation.

F8 Is it portable? DEEP

It's standard C11 plus POSIX threads, and builds cleanly with `-Wall -Wextra -Wpedantic -Werror` on GCC and Clang. `sp_auto` needs GCC/Clang (MSVC doesn't support the cleanup attribute).

F9 What if the program itself runs out of memory? DEEP

Every allocation path handles NULL. We test this with built-in fault injection: `SENTINEL_FAIL_AT=n` makes the n-th allocation fail, and the tests check that nothing leaks or crashes.

Part G · Testing and proof

G1 How do you know it works? CORE

> **Say:** "*58 automated tests in 8 suites. Each test causes a situation on purpose, like a double free, and checks the result and the issue counter. Then three independent tools check our code: Valgrind finds 0 errors and 0 leaks, AddressSanitizer with UBSan is clean, and ThreadSanitizer finds no races. And everything builds with warnings treated as errors.*"

G2 How can a test check a double free if your library prints instead of crashing? DEEP

The tests run in `SENTINEL=silent` mode and read counters: `sp_guard_issues(SP_ISSUE_DOUBLE_FREE)` must go up by exactly 1. The exit-time analysis is tested by running the program as a child process with a JSON log and checking the JSON.

G3 Your demo shows printed messages. How do we know they aren't fake? TRICKY

- Run **Valgrind** on both: plain C shows "Invalid free, 1 error"; ours shows 0 errors.
- **Break it on purpose:** comment out one line in the library, and the tests fail immediately. Restore it, and they pass.
- Let him **change the demo** (delete a free): the leak report appears with that line.

Cheat sheet: one-line answers

If he asks…	Say…
How does normal malloc go through you?	Macros in smartptr.h rewrite the calls into ours
How is the line number known?	`__FILE__`, `__LINE__`, `__func__` captured by the macro
How is a double free stopped?	The record says FREED, so we never call the real free again
How is overflow detected?	16 known sentinel bytes after each block
What is freezing / quarantine?	Freed memory is filled with 0xDD and held, so misuse shows up
Hierarchy of freeing?	Destructors free children first, then parent; ownership decides, not position
Shared child?	Survives its parent until its last owner drops it
Parent back-pointer?	Weak, so no cycle; cycles are detected anyway
Cycle detection?	Conservative scan + Tarjan's SCC at exit
Leak detection?	atexit handler: records still LIVE = leaks, grouped by line
Thread safety?	One mutex, released during destructors; TSan clean
Cost?	~30× slower malloc; a development tool

Don't say these (they're not true)

- ✗ "It fixes every bug automatically". It prevents some, corrects some, and detects and reports the rest (Part B2).
- ✗ "It's a garbage collector". It isn't (F3).
- ✗ "It frees struct members automatically in plain C". Only with a destructor (D5).
- ✗ "It detects every memory error". Reads after free and underflows aren't detected (F6).
- ✗ "It has zero cost". It's about 30× slower malloc (F4).